Ders 11 Çalışma Özeti

Ders 11 Çalışma Özeti

Genel Konular

- Large Class (aşırı büyük sınıf)
  - Hoca “large” yerine “aşırı büyük” çevirisini seçti: büyüklük tek başına sorun değildir, sorun aşırılıktır.
  - Kök neden: sınıfa çok sayıda ve birbiriyle ilgisiz sorumluluk atanmasıdır. Sorumluluk sayısı arttıkça aralarındaki ilgi azalır ve uyum (cohesion) düşer (ekrandan, 00:10:00).
  - Ölçüler: ilk akla gelen kod satırı sayısıdır (LOC), ama üye alan sayısının fazlalığı da bir göstergedir. Fazla sorumluluk verilen sınıfa gerekli veriler eklenir, veri çok olunca yeni işler de o sınıfa verilir; sınıf kartopu gibi büyüyen bir döngüye girer.
  - Nesneye dayalı yaklaşımın önerdiği “böl, parçala, yönet” stratejisinin tersidir. İyi tasarım, problem alanındaki varlıkları en az onlar kadar ince parçalara ayırır.
  - Çözüme götüren düzenlemeler (slayt 31): Extract Class, Extract Subclass, Duplicate Observed Data.
- Extract Class (sınıf çıkartma)
  - Birbiriyle ilgisiz iki iş yapan bir sınıfı bu işlere göre ikiye ayırmak; ikinci işle ilgili üyeleri seçip yeni bir sınıfa taşımak (ekrandan, 00:11:45).
  - Birden fazla kod kusurunu düzeltir: Large Class’ın yanında Long Method’u da. Extract Method ile çıkarılan metodun “uygun yeri” yepyeni bir sınıfsa yapılan iş Extract Class olur.
  - Örnek: `Person` sınıfında `name`, `officeAreaCode`, `officeNumber` ve `getTelephoneNumber` var. Sonra: `Person` (`name`, `getTelephoneNumber`) → `officeTelephone` ilişkisiyle (1) → `Telephone Number` (`areaCode`, `number`, `getTelephoneNumber`).
  - Cep telefonu gibi yeni numaralar ikinci bir `TelephoneNumber` üyesiyle eklenir; telefon verisi üzerinde operatör analizi gibi işler de ayrı sınıfta daha rahat yapılır.
- Extract Subclass (alt sınıf çıkartma)
  - Sınıfın bazı üyeleri her zaman kullanılmıyorsa bu üyelerden yeni bir alt sınıf oluşturmak (ekrandan, 00:14:30).
  - İpucu: metotlarda if-else ya da alan değerlerine göre “şu durumda şu alanları, öbür durumda bu alanları kullan” gibi ayrık erişim desenleri.
  - Sözlü örnek: maaş hesabında yönetici ise ikramiye eklenir; yönetici sınıfı çalışanın alt sınıfı olur.
  - Slayt örneği: `Job Item` (`getTotalPrice`, `getUnitPrice`, `getEmployee`) → `Job Item` (`getTotalPrice`, `getUnitPrice`) ve ondan kalıtımla `Labor Item` (`getUnitPrice`, `getEmployee`). Bazı işlerde işi kimin yaptığı önemlidir, bazılarında değildir.
- Duplicate Observed Data (gözlenen verinin çoğullanması)
  - Uygulama alanı verisi yalnız bir GUI sınıfında saklanmamalı; bilgi katmanında yeni bir sınıfa taşınmalı ve GUI nesnesiyle bu bilgi nesnesi arasında Observer kalıbı kurulmalıdır (ekrandan, 00:20:00).
  - Örnek: `Interval Window` içinde `startField`, `endField`, `lengthField` (TextField), odak kaybı olay metotları ve `calculateLength`, `calculateEnd` vardır. Sonra: `Interval Window` `Observer` arayüzünü gerçekler ve `Interval` sınıfına (1) bağlanır; `Interval` (`start`, `end`, `length`: String; `calculateLength`, `calculateEnd`) `Observable`’dan türer.
  - Kazanç: GUI değişse, başka platforma geçilse ya da veri bir servisten gelse bile iş mantığı değişmez.
- Divergent Change ve Shotgun Surgery (değişikliklerin farklı noktalarda yapılmasının gerekmesi)

  - Gereksinim değişikliği ya da hata düzeltmesi için kod değişecektir; tasarım ne kadar iyiyse değişiklik o kadar az yerde yapılır (ekrandan, 00:22:55).
  - Divergent Change: bir değişiklik için gereken düzenlemeler aynı sınıfın birçok metodunda yapılır. Örnek: yeni bir veritabanına geçmek için bir sınıfın 3 metodu değişir.
  - Shotgun Surgery: düzenlemeler birçok sınıfa dağılır, her sınıfta az sayıda metot değişir. Örnek: yeni bir tür kredi için 2 sınıftaki 5 metot değişmeli.
  - Çözüm: Extract Class; Move Method / Move Field (özellikle Shotgun Surgery'de). Değişikliklerin toplandığı yeni sınıfla eski sınıflar arasında sahiplik, parça-bütün ya da kullanma ilişkisi kurulur.
- Move Method (metot taşınması)
  - Metot tanımlandığı sınıftan çok başka bir sınıftan çağrılıyorsa en çok çağrıldığı sınıfa taşınır; kullandığı alanlar da taşınabilir (ekrandan, 00:30:00).
  - Eski sınıftaki metot ya silinir ya da yeni yere çağrı ileten (delegation) bir metot olarak kalır. Silmek API'yi değiştirdiği için refactoring'in geriye uyumluluk hedefine aykırıdır; çağrı iletme tercih edilir.
  - Extract Class gibi başka düzenlemelerin alt adımıdır, ama hoca başlı başına bir düzenleme olmayı hak ettiğini söyledi.
- Primitive Obsession (ilkel tiplerin aşırı kullanımı)
  - Yapısal programlamadan gelenler ilkel tipleri aşırı yoğun kullanır (ekrandan, 00:30:25).
  - Çözüm düzenlemeleri: Replace Data Value with Object, Replace Type Code with Class, Replace Type Code with Subclasses, Replace Type Code with State/Strategy, Replace Array with Object; ayrıca Extract Class ve Introduce Parameter Object.
- Replace Data Value with Object
  - Bir üye alan, hele onunla ilgili davranışlar varsa, bir sınıfla simgelenmelidir (ekrandan, 00:32:35).
  - Örnek: `Order` içinde `customer: String` → ayrı `Customer` (`name: String`) sınıfı ve `Order` ile ilişki. Müşteri adını string tutmak, nesnelerde verimsiz bir "yabancı anahtar" gibi tüm müşterileri tarayıp ad eşleştirmeyi gerektirir.
  - Müşterinin tüm siparişlerini aramak gibi işler çoğaldıkça müşteri sınıfı ayrılmazsa sipariş sınıfı aşırı büyür.
- Replace Type Code with Class
  - Sabit değişken kullanma alışkanlığıyla ilgilidir (ekrandan, 00:36:40).
  - Örnek: `Person` içinde statik `O`, `A`, `B`, `AB` (int) sabitleri ve `bloodGroup: int`. `bloodGroup`'a 8 atanabilir; anlamsız değerlere kapı açılır. Sonra: `Person` → (1) `BloodGroup`; `BloodGroup` içinde statik `O`, `A`, `B`, `AB` alanları `BloodGroup` türündendir.
  - Kısaca `enum` kullanılabilir (Rh faktörü, müşteri kategorisi gibi her bilgi için ayrı enum).
- Replace Type Code with Subclasses
  - Sınıfın davranışını belirleyen ilkel tip kodu yerine kalıtım daha uygundur (ekrandan, 00:39:45).
  - Belirti: her metotta tip koduna göre `switch` ya da uzun if-else zinciri.
  - Örnek (ekrandan, 00:42:40): `Employee` içinde `_type` ve `ENGINEER = 0`, `SALESMAN = 1`, `MANAGER = 2`; `work()` içinde `switch`. Sonra: soyut `Employee` ve `work()`'ü kendi biçiminde gerçekleyen `Engineer`, `Salesman` alt sınıfları.
  - `switch` içinde `default` varsa `Employee` soyut yapılmaz. Replace Conditional with Polymorphism'e benzer, onun özelleşmiş halidir.
- Replace Type Code with State/Strategy
  - Kalıtım bir nedenle mümkün değilse (alt sınıfın başka bir sınıftan türemesi gerekiyorsa) State ya da Strategy kalıbı kullanılır (ekrandan, 00:45:10).
  - `Employee` → (1) `Employee Type`; `Engineer` ve `Salesman` `Employee Type`'tan türer. `Employee.work()` tür nesnesinin `work()`'ünü çağırır.
  - Daha esnektir: mühendis satışa geçince tek bir atamayla rolü değişir; nesne silinip yeniden oluşturulmaz.
- Replace Array with Object
  - Çok boyutlu dizilerde ya da paralel tek boyutlu dizilerde kayıt tutmak yerine tek bir nesne kullanılır (ekrandan, 00:50:00).
  - Örnek: `String[][] scores` içinde takım adı ve sayısal değerler string olarak tutulur → `Score` nesnelerinden oluşan dizi.
- Feature Envy (özellik kıskançlığı)

  - Bir metot ait olduğu sınıfın üyelerinden çok başka sınıfların üyelerine erişiyorsa (ekrandan, 00:50:50).
  - "Çok": koddaki ortalamanın ya da %20 gibi bir eşiğin üstü. Sorumlulukların doğru atanmadığının göstergesi olabilir.
  - Sıfır bağlaşım olamaz; hiçbir sınıf yalnız kendi metotlarını kullanmaz.
  - Strategy ve Visitor kalıpları özellik kıskançlığı sayılmaz.
  - Çözüm: Move Method; kıskançlık metodun yalnız bir kısmındaysa Extract Method ile o kısım çıkarılıp kıskanılan sınıfa taşınır.

Hocanın Özellikle Vurguladığı Kısımlar

- Large Class'ın kök nedeni sorumlulukların aşırı fazlalığıdır.
  - Sınıf büyüdükçe uyum düşer; Extract Class ve Extract Subclass genelde çözüme götürür.
- Sınıfları bölmek toplam kodu uzatabilir ama birim başına küçültür.
  - 1000 satırlık `Interval Window` bölününce örneğin 600 ve 700 satırlık iki sınıf (toplam 1300) çıkabilir. Bakım sırasında tek bir sınıf üzerinde çalışıldığından "daha kolay yutulan lokmalar" elde edilir.
- Divergent Change ile Shotgun Surgery farkı:
  - Değişiklikler tek sınıfın birçok metodunda toplanıyorsa Divergent Change, birçok sınıfa saçılıyorsa Shotgun Surgery. Hoca, Visitor örneğinin sınıf şemasında değişiklik noktalarını işaretleyerek farkı gösterdi: yeşiller toplu, kırmızılar "çifteyle ateş edilmiş gibi" dağınık (ekrandan, 00:25:00).
- Move Method'da eski metot silinmemeli, çağrı iletmelidir.
  - Refactoring'in en önemli hedeflerinden biri geriye uyumluluktur: dışa açılan arayüz değişmeden iç yapı değişir.
- Strategy çözümü kalıtım çözümünden daha esnektir.
  - Rol değişimi tek bir atamayla yapılır.
- Strategy ve Visitor'da başka nesnenin durumunu sorgulamak Feature Envy değildir.
  - O işin yeni sınıfın nesnesine yaptırılması zaten istenmiştir; gerekli bilgiyi alması kaçınılmazdır.
- Tasarım kalıpları ile kod kusurları iyi tasarımın iki yüzüdür.
  - Kalıplar bir şeyin nasıl iyi yapıldığını, kod kusurları bir şeyin nerede yanlış olduğunu ve nasıl düzeltileceğini öğretir; ikisi birlikte bilinmelidir.

Kısa Tekrar Notları

- Large Class → Extract Class, Extract Subclass, Duplicate Observed Data.
- Extract Class: ilgisiz ikinci işin üyeleri yeni sınıfa; `Person` → `TelephoneNumber`.
- Extract Subclass: her zaman kullanılmayan üyeler alt sınıfa; `JobItem` → `LaborItem`.
- Duplicate Observed Data: alan verisini GUI'den ayır, Observer ile eşgüdüm; `IntervalWindow` + `Interval`.
- Divergent Change: tek sınıf, çok metot. Shotgun Surgery: çok sınıf, az metot. Çözüm: Extract Class, Move Method/Field.
- Move Method: en çok çağrıldığı sınıfa taşı, eski yerde çağrı ilet.
- Primitive Obsession → Replace Data Value with Object, Replace Type Code with Class / Subclasses / State-Strategy, Replace Array with Object.
- Tip koduna göre `switch` görülüyorsa alt sınıf ya da Strategy; `default` varsa üst sınıf soyut yapılmaz.
- Feature Envy: başka sınıfların üyelerine eşikten fazla erişim; Move Method ya da Extract Method. Strategy ve Visitor sayılmaz.

Detaylı Açıklamalar

- Large Class ve çözümleri
  - Bu ders, önceki mevcut kayıtta (ders 9) başlayan kod kusurları dizisini sürdürür. Aşırı büyük sınıfta sorumluluklar birbirinden kopuktur; uyum ilk haftalarda konuşulan iki temel kalite ölçüsünden biridir (bağlaşım ve uyum). Sınıfın büyüklüğü satır sayısıyla

ölçülebilir, ama üye alan sayısı da göstergedir; veri biriktikçe sınıfa yeni işler verilir ve döngü büyür.
  - Extract Class'ta yeni sınıfa yalnız veriler değil, o verilerle çalışan metotlar da taşınır; kusurlu sınıf ne kadar büyükse ondan o kadar çok sınıf çıkarılabilir. `Person` örneğinde telefonla ilgili alanlar ayrı `TelephoneNumber` sınıfına gider ve `Person` bu sınıfla sahiplik ilişkisi kurar. Bu alanlar kendi aralarında sıkı, `Person`'ın diğer üyeleriyle gevşek ilişkilidir; bu, ayırmanın işaretidir.
  - Extract Subclass'ta çıkarılan sınıf mevcut sınıfın alt sınıfıdır. `LaborItem` ("emek" vurgusuyla) belirli bir çalışanın yaptığı işi modeller ve ek olarak `employee` üyesi ile `getEmployee` metoduna sahiptir.
  - Duplicate Observed Data, iş mantığı ile sunum katmanı iç içe geçip "ip yumağı" olduğunda kullanılır. Giriş ekranları tutarlılık denetimi gibi iş kurallarını içermeye eğilimlidir. Katmanlı tasarımda bir katmandaki değişiklik diğer katmanları etkilemez ya da en az etkiler. `Interval Window` örneğinde başlangıç, bitiş ve süre alanlarından ikisi verilince üçüncüsü hesaplanır; bu iş mantığı `Interval` sınıfına geçer ve `Interval` GUI ile ilgili hiçbir şey içermez.
- Divergent Change ve Shotgun Surgery
  - Hoca iki adı esprili buldu: "shotgun", saçma atan çifte tüfektir; Shotgun Surgery "çifteyle ameliyat" gibi, değişikliklerin kodun her yerine saçılmasıdır. Divergent Change için sözlü örnekte bir sınıfın 3 metodunda toplam 8 değişiklik, Shotgun Surgery için 2 sınıfta 5 ya da 3 sınıfta 8 metot verildi.
  - Visitor örneğinin şemasında (ProjectItem, Deliverable, Task, Project, Contact, ProjectVisitor, ProjectCostVisitor, ProjectTimeVisitor, Report) hoca dağınık sınıflardaki metot ve alanları kırmızıyla, `ProjectVisitor`'ın visit metotları ve `ProjectCostVisitor` gibi toplu bir bölgedeki değişiklikleri yeşille işaretledi (ekrandan, 00:25:00).
- Move Method
  - Metot, tanımlandığı sınıftan çok başka bir sınıftan çağrılıyorsa yanlış yere konmuştur. Slayttaki örnekte `Class 1`'deki `aMethod()` `Class 2`'ye taşınır (ekrandan, 00:30:00). Hoca, `Class 2`'nin 8 ayrı metodu bu metodu çağırıyorsa metodun baştan oraya konması gerektiğini söyledi. Extract Class'ın alt adımı olsa da yeni sınıf çıkarmadan mevcut sınıflar arasında da yapılabildiği için ayrı bir düzenlemedir.
- Primitive Obsession ve "Replace ... with ..." düzenlemeleri
  - Hoca "obsession"ı "takıntı" yerine "aşırı kullanım" diye çevirdi. Kusurun farklı biçimlerini tek tek tarif etmek yerine her biçimi kendi düzenlemesiyle inceledi.
  - Replace Data Value with Object: `Order` içindeki müşteri adı string olarak durursa, en çok sipariş veren 100 müşterinin iletişim bilgileri gibi bir rapor için bütün müşteri kayıtları taranıp ad eşleştirilir. Bu, veri tabanı odaklı tasarımdan kalan yabancı anahtar kullanımıdır; veri tabanında verimli olsa da nesnelerde verimsizdir. Sipariş nesnesinden müşteri tek adımda nesne olarak alınabilmelidir.
  - Replace Type Code with Class: kan grubu için dört statik sabit (O, A, B, AB) ve bir `int` alan kullanılırsa arka planda 0, 1, 2, 3 gibi değerler atanır; alana 8 atanması engellenmez. Ayrı `BloodGroup` sınıfında her grup bu sınıf türünden statik bir alandır ve `Person` ile birebir sahiplik ilişkisi kurulur. Birçok dilde bunun hazır altyapısı `enum`'dur. Hoca enum'u herkesin bildiğini varsayıp enum kodunu gösteren iki slaytı atladı.
  - Replace Type Code with Subclasses: çalışan türleri (mühendis, satış elemanı, hizmet elemanı, çağrı merkezi elemanı gibi) sabitlerle tutulursa maaş, ikramiye gibi her metotta `switch` ya da `if type == ENGINEER ... else if ...` zinciri olur. Bu Long Method ve Large Class'a da yol açar. Türler `Employee`'nin alt sınıfları yapılır ve her `case`'in içeriği ilgili alt sınıfın metoduna taşınır. Böylece `Engineer`'ın çalışma biçimi değişince `Salesman` ve diğer sınıflara dokunulmaz. Rastgele bir "çalışan" nesnesine gerek yoksa `Employee` soyut yapılır.
  - Replace Type Code with State/Strategy: kalıtım zinciri istenmiyorsa ya da alt sınıfın başka bir sınıftan türemesi daha önemliyse kullanılır. `Employee` bir `EmployeeType` üyesine sahip olur; somut tür (`Engineer`, `Salesman`, `Manager`) bu üyeye atanır. Kalıtım çözümünde mühendisi satışa almak için yeni bir `Salesman` nesnesi oluşturup durum bilgisini aktarmak ve eski nesneyi silmek gerekir; Strategy çözümünde tek bir atama yeterlidir.
  - Replace Array with Object: hoca, veri yapısı (struct) görmemiş birinci sınıf öğrencilerine farklı türde verileri tutmanın bir yolu olarak dizilerin gösterildiğini, bazılarının bu alışkanlığı sürdürdüğünü söyledi. İki boyutlu string dizisinde sayılar bile string

tutulur; paralel üç dizide (`teamNames`, `points`, `average`) aynı indis aynı takımdır ama veri dağınıktır ve kolay unutulur. Çözüm, üyeleri takım adı ve sayısal değerler olan bir `Score` sınıfı ve bu türden bir nesne dizisidir.

- Feature Envy
  - Daha önce görülen Move Method'da bir üyenin başka sınıflardan çok kullanılması söz konusuydu; Feature Envy olaya tersten bakar: bir metot başka sınıfların üyelerine kendi sınıfınınkinden çok erişir. Eşik, proje ortalamasından ya da başarılı sayılan önceki projelerin ortalamasından belirlenebilir; hoca sözlü olarak ortalamanın en fazla %10 üstüne hata payı tanınabileceğini, slaytta da %20 gibi bir eşiği verdi.
  - Çözüm, metodu en çok kıskandığı sınıfa taşımaktır. Kıskançlık metodun tümünde değil bir kısmındaysa yalnız o kısım çıkarılıp taşınır.
- Ders, Fowler'ın kitabından seçilmiş, kendi içinde en yoğun ilişkili kod kusuru ve refactoring kümesini bitirdi. Slaytın son kısmı, bu düzenlemelerin birlikte kullanılarak yapısal izler taşıyan bir tasarımın nasıl değiştirildiğine örnek olarak Fowler'ın Refactoring kitabının 1. bölümünü gösterir (ekrandan, 00:50:50); hoca bu bütüncül örneğin sonraki derste adım adım yapılacağını söyledi.
- Ders sonrası notlar
  - Slayt 43'teki `public abstract void work(){};` Java'da derlenmez ("abstract methods cannot have a body"); doğrusu `public abstract void work();`. Aynı slayttaki `extends Employ` de `extends Employee` olmalıdır (ajan tarafından `java` ile denendi).
  - Slayt 45'teki `Score[teamCount] scores;` Java'da geçerli bir bildirim değildir; doğrusu `Score[] scores = new Score[teamCount];` (ajan tarafından `java` ile denendi).
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.